

SOAL	JAWABAN
[WOS 20] A median is defined as a number separating the higher half of a data set from the lower half. Query the median of the Northern Latitudes (LAT_N) from STATION and round your answer to 4 decimal places. Input Format The STATION table is described as follows:	<pre>WITH cte AS (SELECT LAT_N, ROW_NUMBER() OVER (ORDER BY LAT_N) AS rn, COUNT(*) OVER () AS n FROM STATION) SELECT ROUND(AVG(LAT_N), 4) FROM cte WHERE rn IN (FLOOR((n + 1) / 2), FLOOR((n + 2) / 2));</pre>
[WOS 19] Consider $P_1(a, c)$ and $P_2(b, d)$ to be two points on a 2D plane where (a, b) are the respective minimum and maximum values of Northern Latitude (LAT_N) and (c, d) are the respective minimum and maximum values of Western Longitude (LONG_W) in STATION. Query the Euclidean Distance between points P_1 and P_2 and format your answer to display 4 decimal digits.	<pre>SELECT ROUND(SQRT(POWER(MAX(LAT_N) - MIN(LAT_N), 2) + POWER(MAX(LONG_W) - MIN(LONG_W), 2)), 4) FROM STATION;</pre>
[WOS 18] Consider $P_1(a, b)$ and $P_2(c, d)$ to be two points on a 2D plane. a happens to equal the minimum value in Northern Latitude (LAT_N in STATION). b happens to equal the minimum value in Western Longitude (LONG_W in STATION). c happens to equal the maximum value in Northern Latitude (LAT_N in STATION). d happens to equal the maximum value in Western Longitude (LONG_W in STATION). Query the Manhattan Distance between points P_1 and P_2 and round it to a scale of 4 decimal places.	<pre>SELECT ROUND ((ABS(MAX(LAT_N) - MIN(LAT_N)) + ABS(MAX(LONG_W) - MIN(LONG_W))), 4) FROM STATION;</pre>
[WOS 17] Query the <i>Western Longitude (LONG_W)</i> where the smallest <i>Northern Latitude</i>	<pre>SELECT ROUND(LONG_W, 4) FROM STATION</pre>

(LAT_N) in STATION is greater than . Round your answer to decimal places.	WHERE LAT_N > 38.7780 ORDER BY LAT_N LIMIT 1;
[WOS 16] Query the smallest <i>Northern Latitude</i> (LAT_N) from STATION that is greater than . Round your answer to decimal places.	SELECT ROUND(LAT_N, 4) FROM STATION WHERE LAT_N > 38.7780 ORDER BY LAT_N LIMIT 1;
[WOS 15] Query the <i>Western Longitude</i> (LONG_W) for the largest <i>Northern Latitude</i> (LAT_N) in STATION that is less than . Round your answer to decimal places.	SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N < 137.2345 ORDER BY LAT_N DESC LIMIT 1;
[WOS 14] Query the greatest value of the <i>Northern Latitudes</i> (LAT_N) from STATION that is less than . Truncate your answer to decimal places.	SELECT ROUND(MAX(LAT_N), 4) FROM STATION WHERE LAT_N < 137.2345;
[WOS 13] Query the sum of <i>Northern Latitudes</i> (LAT_N) from STATION having values greater than and less than . Truncate your answer to decimal places.	SELECT ROUND(SUM(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7880 AND LAT_N < 137.2345;
[WOS 12] Query the list of <i>CITY</i> names from STATION that <i>do not start</i> with vowels and <i>do not end</i> with vowels. Your result cannot contain duplicates.	SELECT DISTINCT CITY FROM STATION WHERE LOWER(CITY) NOT REGEXP '^.*[aiueo]\$' AND LOWER(CITY) NOT REGEXP '^[aioeu].*\$';
[WOS 11] Query the list of <i>CITY</i> names from STATION that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.	SELECT DISTINCT CITY FROM STATION WHERE UPPER(CITY) NOT REGEXP '^[AIUEO].*\$' OR UPPER(CITY) NOT REGEXP '^.*[AIUEO]\$';
[WOS 10] Query the list of <i>CITY</i> names from STATION that <i>do not end</i> with vowels. Your result cannot contain duplicates.	SELECT DISTINCT CITY FROM STATION WHERE LOWER(CITY) NOT REGEXP '^.*[aieuo]\$';
[WOS 9] Query the list of <i>CITY</i> names from STATION that <i>do not start</i> with vowels. Your result cannot contain duplicates.	SELECT DISTINCT CITY FROM STATION WHERE LOWER(CITY) NOT REGEXP '^[aieuo].*\$';
[WOS 8] Query the list of <i>CITY</i> names from STATION which have vowels (i.e., a, e, i, o, and u) as both their first <i>and</i> last characters. Your result cannot contain duplicates.	SELECT DISTINCT CITY FROM STATION WHERE LOWER(CITY) REGEXP '^[aeiou].*[aeiou]\$';
[WOS 7] Query the list of <i>CITY</i> names ending with vowels (a, e, i, o, u) from STATION . Your result <i>cannot</i>	SELECT DISTINCT CITY FROM STATION

contain duplicates.	<pre>WHERE UPPER(CITY) REGEXP '^.*[AIUEO]\$';</pre>
[WOS 6] Query the list of <i>CITY</i> names starting with vowels (i.e., a, e, i, o, or u) from STATION . Your result <i>cannot</i> contain duplicates.	<pre>SELECT DISTINCT CITY FROM STATION WHERE UPPER(CITY) REGEXP '^[AIUOE].*\$';</pre>
[WOS 5] Query the two cities in STATION with the shortest and longest <i>CITY</i> names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.	<pre>SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY), CITY LIMIT 1; SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY) DESC, CITY LIMIT 1;</pre>
[WOS 4] Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table.	<pre>SELECT COUNT(CITY) - COUNT(DISTINCT CITY) FROM STATION;</pre>
[WOS 3] Query a list of CITY names from STATION for cities that have an even ID number. Print the results in any order, but exclude duplicates from the answer.	<pre>SELECT DISTINCT CITY FROM STATION WHERE MOD(ID,2) = 0;</pre> Alternatif solusi: <pre>SELECT DISTINCT CITY FROM STATION WHERE ID % 2 = 0;</pre>
[WOS 2] Query the following two values from the STATION table: 1. The sum of all values in <i>LAT_N</i> rounded to a scale of decimal places. 2. The sum of all values in <i>LONG_W</i> rounded to a scale of decimal places	<pre>select round(sum(lat_n), 2) as lat, round(sum(long_w),2) as lon from station;</pre>
[Higher Than 75 Marks] Query the <i>Name</i> of any student in STUDENTS who scored higher than <i>Marks</i> . Order your output by the <i>last three characters</i> of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending <i>ID</i> .	<pre>SELECT NAME from STUDENTS where MARKS > 75 ORDER BY RIGHT(NAME,3) , ID ;</pre>
[Employee Names] Write a query that prints a list of employee names (i.e.: the <i>name</i> attribute) from the Employee table in alphabetical order.	<pre>SELECT NAME FROM EMPLOYEE ORDER BY NAME;</pre>
[Employee Salaries] Write a query that prints a list of employee names (i.e.: the <i>name</i> attribute) for employees in Employee having a salary greater than per month who have been employees for less than months. Sort your result by ascending <i>employee_id</i> .	<pre>SELECT NAME FROM EMPLOYEE WHERE SALARY > 2000 AND MONTHS < 10 ORDER BY EMPLOYEE_ID;</pre>
[African Cities]	<pre>SELECT CITY.NAME FROM CITY JOIN</pre>

Given the CITY and COUNTRY tables, query the names of all cities where the <i>CONTINENT</i> is 'Africa'. Note: <i>CITY.CountryCode</i> and <i>COUNTRY.Code</i> are matching key columns.	<pre>COUNTRY ON CITY.COUNTRYCODE = COUNTRY.CODE WHERE COUNTRY.CONTINENT = 'Africa';</pre>
[Average Population of Each Continent] Given the CITY and COUNTRY tables, query the names of all the continents (<i>COUNTRY.Continent</i>) and their respective average city populations (<i>CITY.Population</i>) rounded down to the nearest integer.	<pre>SELECT COUNTRY.CONTINENT, FLOOR(AVG(CITY.POPULATION)) FROM CITY JOIN COUNTRY ON CITY.COUNTRYCODE = COUNTRY.CODE GROUP BY COUNTRY.CONTINENT;</pre> The keyword is 'round down' in the qn. floor - rounds down always round - might round up
[The Report] Ketty gives Eve a task to generate a report containing three columns: <i>Name</i>, <i>Grade</i> and <i>Mark</i>. Ketty doesn't want the NAMES of those students who received a grade lower than 8. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.	<pre>SELECT CASE WHEN G.GRADE >= 8 THEN S.NAME ELSE NULL END AS NAME, G.GRADE, S.MARKS FROM STUDENTS S, GRADES G WHERE S.MARKS <= MAX_MARK AND S.MARKS >= MIN_MARK ORDER BY G.GRADE DESC, S.NAME ASC, S.MARKS ASC;</pre>
[Top Competitors] Julia baru saja menyelesaikan sebuah kontes coding dan membutuhkan bantuan untuk menyusun leaderboard. Terdapat empat tabel, yaitu Hackers yang berisi hacker_id dan name, Difficulty yang berisi difficulty_level dan score sebagai skor maksimum untuk setiap tingkat kesulitan, Challenges yang berisi challenge_id, hacker_id pembuat challenge, dan difficulty_level, serta Submissions yang berisi submission_id, hacker_id peserta yang mengirim submission, challenge_id, dan score yang diperoleh. Tugasmu adalah menampilkan hacker_id dan name dari hacker yang berhasil memperoleh full score pada lebih dari satu challenge, dengan ketentuan full score terjadi ketika score pada tabel Submissions sama dengan score maksimum pada tabel Difficulty sesuai difficulty_level challenge tersebut. Urutkan hasil berdasarkan jumlah challenge full score terbanyak secara descending, dan jika terdapat jumlah yang sama, urutkan berdasarkan hacker_id secara ascending.	<pre>SELECT H.HACKER_ID, H.NAME FROM CHALLENGES C JOIN DIFFICULTY D ON C.DIFFICULTY_LEVEL = D.DIFFICULTY_LEVEL JOIN SUBMISSIONS S ON C.CHALLENGE_ID = S.CHALLENGE_ID JOIN HACKERS H ON S.HACKER_ID = H.HACKER_ID WHERE D.SCORE = S.SCORE GROUP BY H.HACKER_ID, H.NAME HAVING COUNT(DISTINCT S.CHALLENGE_ID) > 1 ORDER BY COUNT(DISTINCT S.CHALLENGE_ID) DESC, H.HACKER_ID ;</pre>
[Type of Triangle] Write a query identifying the <i>type</i> of each record in the TRIANGLES table using its three side lengths. Output	<pre>SELECT CASE WHEN A + B <= C OR A + C <= B OR B + C</pre>

one of the following statements for each record in the table: • Equilateral: It's a triangle with sides of equal length. • Isosceles: It's a triangle with sides of equal length. • Scalene: It's a triangle with sides of differing lengths. • Not A Triangle: The given values of <i>A</i>, <i>B</i>, and <i>C</i> don't form a triangle.	<pre><= A THEN 'Not A Triangle' WHEN (A = B AND B = C) THEN "Equilateral" WHEN (A = B) OR (A = C) OR (B = C) THEN "Isosceles" ELSE "Scalene" END FROM TRIANGLES;</pre>
Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary. Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.	<pre>SELECT CEIL(AVG(SALARY*1.0) - AVG(REPLACE(SALARY, '0', '')*1.0)) FROM EMPLOYEES;</pre> Ceil untuk round up ke integer Replace untuk hilangkan 0
[Top Earners] We define an employee's <i>total earnings</i> to be their monthly worked, and the <i>maximum total earnings</i> to be the maximum total earnings for any employee in the Employee table. Write a query to find the <i>maximum total earnings</i> for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.	<pre>SELECT MONTHS * SALARY AS EARNINGS, COUNT(*) FROM EMPLOYEE GROUP BY MONTHS * SALARY ORDER BY MONTHS * SALARY DESC LIMIT 1;</pre>
[Occupations] Pivot the <i>Occupation</i> column in OCCUPATIONS so that each <i>Name</i> is sorted alphabetically and displayed underneath its corresponding <i>Occupation</i>. The output should consist of four columns (<i>Doctor</i>, <i>Professor</i>, <i>Singer</i>, and <i>Actor</i>) in that specific order, with their respective names listed alphabetically under each column. Note: Print NULL when there are no more names corresponding to an occupation. The first column is an alphabetically ordered list of Doctor names. The second column is an alphabetically ordered list of Professor names. The third column is an alphabetically ordered list of Singer names. The fourth column is an alphabetically ordered list of Actor names. The empty cell data for columns with less than the maximum number of names per occupation (in this case, the Professor and Actor columns) are filled with NULL values.	<pre>SELECT MAX(CASE WHEN occupation = 'Doctor' THEN name END) AS Doctor, MAX(CASE WHEN occupation = 'Professor' THEN name END) AS Professor, MAX(CASE WHEN occupation = 'Singer' THEN name END) AS Singer, MAX(CASE WHEN occupation = 'Actor' THEN name END) AS Actor FROM (SELECT name, occupation, ROW_NUMBER() OVER (PARTITION BY occupation ORDER BY name) AS rn FROM OCCUPATIONS) AS t GROUP BY rn ORDER BY rn;</pre>
[Binary Tree Nodes]	<pre>select n, case when p is null then</pre>

You are given a table, BST, containing two columns: N and P, where N represents the value of a node in Binary Tree, and P is the parent of N.

Column	Type
N	Integer
P	Integer

Write a query to find the node type of Binary Tree ordered by the value of the node. Output one of the following for each node:

- Root: If node is root node.
- Leaf: If node is leaf node.
- Inner: If node is neither root nor leaf node.

```
'Root' when n in (select
distinct p from BST where p is not
null) then 'Inner' else
'Leaf' end as node from BST order by n
asc;
```

[Print Prime Numbers]

Write a query to print all prime numbers less than or equal to **1000**. Print your result on a single line, and use the ampersand (&) character as your separator (instead of a space).

For example, the output for all prime numbers ≤ 10 would be:

```
2&3&5&7
```

```
WITH RECURSIVE nums AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1
    FROM nums
    WHERE n < 1000
)
SELECT GROUP_CONCAT(n ORDER BY n
SEPARATOR '&')
FROM nums n1
WHERE NOT EXISTS (
    SELECT 1
    FROM nums n2
    WHERE n2.n < n1.n
    AND n2.n > 1
    AND MOD(n1.n, n2.n) = 0
);
```

kalau CTE memanggil dirinya sendiri, wajib pakai WITH RECURSIVE.

[Draw The Triangle 2]

```
WITH RECURSIVE bintang AS (
    select 1 as b
    union all
    select b + 1
```

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5): <pre> * * * * * * * * * * * * * * *</pre> Write a query to print the pattern P(20).	<pre> from bintang where b < 20) select repeat('*', b) from bintang order by b asc;</pre>
[Draw The Triangle 1] P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5): <pre> * * * * * * * * * * * * * * *</pre> Write a query to print the pattern P(20).	<pre> WITH RECURSIVE bintang AS (select 1 as b union all select b + 1 from bintang where b < 20) select repeat('*', b) from bintang order by b desc;</pre>
[New Companies] Given the table schemas below, write a query to print the company_code, founder name, total number of lead managers, total number of senior managers, total number of managers, and total number of employees. Order your output by ascending company_code. Note: - The tables may contain duplicate records. - The company_code is string, so the sorting should not be numeric. For example, if the company_codes are C_1, C_2, and C_10, then the ascending company_codes will be C_1, C_10, and C_2.	<pre> select c.company_code, c.founder, count(distinct lm.lead_manager_code), count(distinct sm.senior_manager_code), count(distinct m.manager_code), count(distinct e.employee_code) from Company c join Lead_Manager lm on c.company_code = lm.company_code join Senior_Manager sm on c.company_code = sm.company_code join Manager m on c.company_code = m.company_code join Employee e on c.company_code = e.company_code group by c.company_code, c.founder order by c.company_code;</pre>
[Ollivander's Inventory] Harry Potter and his friends are at Ollivander's with	<pre> with temp_wand as (select w.id, wp.age, w.coins_needed, w.power,</pre>

Ron, finally replacing Charlie's old broken wand. Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each <i>non-evil</i> wand of high power and age. Write a query to print the <i>id</i>, <i>age</i>, <i>coins_needed</i>, and <i>power</i> of the wands that Ron's interested in, sorted in order of descending <i>power</i>. If more than one wand has same power, sort the result in order of descending <i>age</i>.	<pre>row_number() over (partition by power, age order by w.coins_needed) as wand_rank from wands w join wands_property wp on w.code = wp.code where wp.is_evil=0) select id, age, coins_needed, power from temp_wand where wand_rank=1 order by power desc, age desc;</pre>
[Challenges] Julia asked her students to create some coding challenges. Write a query to print the <i>hacker_id</i>, <i>name</i>, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by <i>hacker_id</i>. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.	<pre>WITH challenge_count AS (SELECT h.hacker_id, h.name, COUNT(c.challenge_id) AS total FROM Hackers h JOIN Challenges c ON h.hacker_id = c.hacker_id GROUP BY h.hacker_id, h.name), count_frequency AS (SELECT total, COUNT(*) AS freq FROM challenge_count GROUP BY total), max_count AS (SELECT MAX(total) AS max_total FROM challenge_count) SELECT cc.hacker_id, cc.name,</pre>


```

        cc.total
FROM challenge_count cc
JOIN count_frequency cf
    ON cc.total = cf.total
CROSS JOIN max_count mc
WHERE cc.total = mc.max_total
    OR cf.freq = 1
ORDER BY
    cc.total DESC,
    cc.hacker_id ASC;

```

[Contest Leaderboard]

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of 0 from your result.

```

with carimax as (
    select s.submission_id,
           h.hacker_id,
           h.name, s.challenge_id, s.score,
           row_number() over (
               partition by h.hacker_id, h.name,
                           s.challenge_id
               order by s.score desc) as rn
    from Submissions s
    join Hackers h on h.hacker_id =
                    s.hacker_id
)
select hacker_id, name, sum(score)
from carimax
where rn = 1
group by hacker_id, name
having sum(score) > 0
order by sum(score) desc, hacker_id
asc;

```

[Placements]

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in \$ thousands per month).

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in \$ thousands per month).

```

select s1.Name
from Students s1
join Friends f on s1.ID = f.ID
join Packages p1 on p1.ID = s1.ID
join Students s2 on s2.ID =
f.Friend_ID
join Packages p2 on p2.ID = s2.ID
where p2.Salary > p1.Salary
order by p2.Salary;

```

[Symmetric Pairs]

You are given a table, Functions, containing two columns: X and Y.

Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be symmetric pairs if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such symmetric pairs in ascending order by the value of X. List the rows such that $X_1 \leq Y_1$.

```
with ordered as (  
    select X, Y,  
           row_number () over (partition by  
X, Y order by X, Y) as rn  
    from Functions  
)  
select distinct f1.X, f1.Y  
from ordered f1, ordered f2  
where f1.X <= f1.Y  
and f1.X = f2.Y and f2.X = f1.Y  
and (f1.rn <> f2.rn or f1.X < f1.Y)  
order by X;
```

[SQL Project Planning]

You are given a table, Projects, containing three columns: Task_ID, Start_Date and End_Date. It is guaranteed that the difference between the End_Date and the Start_Date is equal to 1 day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

If the End_Date of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

**Hati-hati fungsi agregasi gaboleh ada spasi.
Misal sum() dengan sum () tuh BEDA banget**

```
with cte as (  
    select Task_ID, Start_Date,  
End_Date,  
           LAG (End_Date) over (order by  
Start_Date) as prev_end_date  
    from Projects  
) ,  
cte2 as (  
    select Start_Date, End_Date,  
           sum (case when Start_Date =  
prev_end_date then 0 else 1 end)  
           over (order by Start_Date) as  
total_group  
    from cte  
) ,  
cte3 as (  
    select  
           min (Start_Date) as project_start,  
           max (End_Date) as project_end,  
           count (*) as total_task  
    from cte2  
    group by total_group  
)  
select project_start, project_end from  
cte3  
order by total_task, project_start;
```

[The PADS]

Generate the following two result sets:

1. Query an alphabetically ordered list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of occurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in ascending order, and output them in the following format:

```
There are a total of [occupation_count]
[occupation]s.
```

where [occupation_count] is the number of occurrences of an occupation in **OCCUPATIONS** and [occupation] is the lowercase occupation name. If more than one Occupation has the same [occupation_count], they should be ordered alphabetically.

```
select concat(Name, '(',
left(Occupation, 1), ')') as query1
from OCCUPATIONS order by Name;
```

```
select concat('There are a total of ',
count(*), ' ',
lower(Occupation), 's.') from
OCCUPATIONS
group by Occupation order by count(*),
Occupation;
```

[]

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the contest_id, hacker_id, name, and the sums of total_submissions, total_accepted_submissions, total_views, and total_unique_views for each contest sorted by contest_id. Exclude the contest from the result if all four sums are 0.

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds 1 screening contest.

Colleges: The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Colleges: The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Challenges: The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

View_Stats: The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

```
with cte as (
    select c.challenge_id,
           sum(coalesce(v.total_views, 0)) as
sumviews,
           sum(coalesce(v.total_unique_views,
0)) as sumuniqviews
    from Challenges c left join
View_Stats v
    on c.challenge_id = v.challenge_id
    group by c.challenge_id
),
cte2 as (
    select h.challenge_id,
           sum(coalesce(s.total_submissions,
0)) as sumsub,
           sum(coalesce(s.total_accepted_submissi
ons, 0)) as sumaccsub
    from Challenges h left join
Submission_Stats s
    on h.challenge_id = s.challenge_id
    group by h.challenge_id
```

Submission_Stats: The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

```
)
select ct.contest_id, ct.hacker_id,
ct.name,
sum(coalesce(cte2.sumsub, 0)),
sum(coalesce(cte2.sumaccsub, 0)),
sum(coalesce(cte.sumviews, 0)),
sum(coalesce(cte.sumuniqviews,0))
from Contests ct
join Colleges cl on ct.contest_id =
cl.contest_id
join Challenges ch on cl.college_id =
ch.college_id
join cte on ch.challenge_id =
cte.challenge_id
join cte2 on ch.challenge_id =
cte2.challenge_id
and cte.challenge_id =
cte2.challenge_id
group by ct.contest_id, ct.hacker_id,
ct.name
having
sum(coalesce(cte2.sumsub, 0)) > 0 or
sum(coalesce(cte2.sumaccsub, 0)) > 0
or
sum(coalesce(cte.sumviews, 0)) > 0 or
sum(coalesce(cte.sumuniqviews,0)) > 0
order by ct.contest_id;
```

[15 Days of Learning SQL]

Julia conducted a 15 days of learning SQL contest. The start date of the contest was March 01, 2016 and the end date was March 15, 2016.

Write a query to print total number of unique hackers who made at least 1 submission each day (starting on the first day of the contest), and find the hacker_id and name of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest hacker_id. The query should print this information for each day of the contest, sorted by the date.

The following tables hold contest data:

```
SELECT
    s.submission_date,

    (
        SELECT COUNT(DISTINCT
s2.hacker_id)
        FROM Submissions s2
        WHERE (
            SELECT COUNT(DISTINCT
s3.submission_date)
            FROM Submissions s3
            WHERE s3.hacker_id =
s2.hacker_id
            AND s3.submission_date
```

- *Hackers*: The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.
- *Submissions*: The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

```

<= s.submission_date
    ) =
DATEDIFF(s.submission_date,
'2016-03-01') + 1
    ) AS total_unique_hackers,

    (
        SELECT s4.hacker_id
        FROM Submissions s4
        WHERE s4.submission_date =
s.submission_date
        GROUP BY s4.hacker_id
        ORDER BY COUNT(*) DESC,
s4.hacker_id ASC
        LIMIT 1
    ) AS hacker_id,

    (
        SELECT h.name
        FROM Hackers h
        WHERE h.hacker_id = (
            SELECT s5.hacker_id
            FROM Submissions s5
            WHERE s5.submission_date =
s.submission_date
            GROUP BY s5.hacker_id
            ORDER BY COUNT(*) DESC,
s5.hacker_id ASC
            LIMIT 1
        )
    ) AS name

FROM (
    SELECT DISTINCT submission_date
    FROM Submissions
) s
ORDER BY s.submission_date;

```